

DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN
ESCUELA DE INGENIERÍA
PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE

IIC2343 - Arquitectura de Computadores (I/2026)

Actividad Formativa 2

Sumador-Restador de 8 bits

Requisitos

- Esta actividad es **formativa**: **no es evaluada** y **no asigna puntaje** a la nota del curso.
- El uso de navegador, equipos electrónicos distintos al entregado por el equipo docente, apuntes externos o cualquier programa distinto a Vivado está **prohibido**. En caso de infracción o sospecha de incumplimiento de este requisito se **expulsará** al estudiante del laboratorio.
- Esta actividad es de carácter **individual** y **presencial**.
- Su desarrollo es en **laboratorio** y deberá ser terminada dentro del horario.
- Deberá ser realizada en VHDL.

Objetivos

- Entender y aplicar el manejo del flujo de datos con `with/select`.
- Entender componentes externos de manejo de entradas y salidas.
- Conocer relojes y registros.
- Entender las tablas de valores de componentes externos.
- Copiar e importar los componentes al proyecto base.
- Incorporar el sumador de 8 bits a un componente que realice las operaciones de suma y de resta.

Materiales Entregados

- Computador con Vivado WebPack instalado.
- Proyecto base de Vivado, con las señales definidas y *constraints* configurados.
- Archivos `.vhd` con los componentes de la actividad: `HA`, `FA`, `Adder8`, `Reg`, `Display_Controller` y `Debouncer`.
- Una placa Basys3.

Alcance de esta actividad

Conviene dejar claro desde el principio qué se pide describir y qué viene resuelto:

- **Se describe:** únicamente la entidad **AdderSub8**, un sumador-restador de 8 bits. Recibe dos operandos, `a` y `b`, y un selector `sel` que elige entre `a + b` y `a - b`; entrega un resultado de 8 bits. Eso es todo su comportamiento.
- **Viene entregado:** el sumador `Adder8` (con sus `FA` y `HA`), el registro `Reg`, el `Display_Controller` y el `Debouncer`. Ya están descritos y no deben modificarse: el trabajo con ellos es **importarlos**, **declararlos** e **instanciarlos**, leyendo su tabla de valores.
- **Se conecta:** el resto del trabajo es cablear los componentes entre sí dentro de `Basys3.vhd` de acuerdo con los diagramas del final del documento.

Criterios de logro (puntaje hipotético)

Importante: esta actividad **no es evaluada** y **no contribuye** a la calificación del curso. La escala siguiente es **solo referencial** (puntaje hipotético): sirve para orientar el nivel de avance esperado y para que el equipo docente comente el logro durante la verificación en placa. No se registra ni se suma a ninguna nota.
La verificación se realiza en la placa Basys3 cargando un operando en `REG A` y otro en `REG B` con los botones, seleccionando la operación con `sw(15)` y leyendo el resultado en el *display* de 7 segmentos. Se podrá retirar tan pronto como una persona del equipo docente lo indique.

- **0,9 puntos hipotéticos (logro completo):** la entidad **AdderSub8** está descrita en VHDL **instanciando el `Adder8` entregado**, y los demás componentes están correctamente conectados; en la placa se pueden cargar los dos operandos, el *display* muestra la suma y la resta según `sw(15)`, y el resultado puede volver a cargarse en un registro para acumular.
- **0,6 puntos hipotéticos (logro parcial):** el circuito se programa en la FPGA y funciona parcialmente de forma observable: por ejemplo, la suma es correcta y la resta entrega `b - a` o le falta el +1 del complemento a dos; o los registros cargan pero el *muxer* de realimentación no permite acumular; o el *display* muestra siempre los operandos y nunca el resultado.
- **0,3 puntos hipotéticos (logro parcial):** el circuito se programa en la FPGA y hay avance verificable (por ejemplo, el *display* responde a los interruptores o un registro carga con su botón), pero ninguna de las dos operaciones entrega el resultado esperado.
- **0 puntos hipotéticos:** en cualquier otro caso (no se logra programar la placa, o los componentes entregados no fueron importados e instanciados).

Descripción

En la Actividad 1 se construyó un sumador por composición de componentes: un *Half-Adder* describe la suma de dos bits con compuertas `xor` y `and`; dos *Half-Adder* forman un *Full-Adder*, que agrega el acarreo de entrada; y encadenando *Full-Adder* se obtiene un sumador del ancho que se quiera. El `Adder8` que se entrega en esta actividad es exactamente eso: ocho `FA` en cascada, donde el acarreo de salida de un bit alimenta el acarreo de entrada del siguiente.
Lo que falta para tener una calculadora es **restar**. La observación central de esta actividad es que **no hace falta un circuito nuevo**: el mismo sumador puede restar si se le preparan bien las entradas.

Flujo de datos: `with/select`

La lógica del componente requiere, de acuerdo con su tabla de valores, poder controlar el flujo de los datos para cada operación. Para seleccionar el flujo de datos se usan componentes como *muxers*, *demuxers*, *encoders* y *decoders*, que en VHDL se expresan con una o más expresiones **`with/select`**:

```
-- Muxer
 with mux_select select
    mux_out <= mux_in_1 when "00",
               mux_in_2 when "01",
               mux_in_3 when "10",
               mux_in_4 when others;
```

Sobre esta construcción conviene tener presente que:

- Todos los valores posibles del selector deben quedar cubiertos. El uso de `when others` permite no tener que escribir todas las opciones y evita que la síntesis infiera memoria no deseada.
- Todas las alternativas deben tener el mismo ancho que la señal asignada.
- La expresión es **concurrente**: describe un circuito permanente, no una decisión que se evalúa una sola vez.

Restar con un sumador: complemento a dos

En **complemento a dos**, restar equivale a sumar el opuesto, y el opuesto de un número se obtiene invirtiendo todos sus bits y sumando 1:

$$a - b = a + (\bar{b} + 1)$$

Ese $+1$ no necesita un sumador adicional: basta con entregar `'1'` como acarreo de entrada (`ci`) del `Adder8`. Y la inversión de `b` es simplemente `not b`, que sobre un vector opera bit a bit. Es decir, ambas operaciones comparten el mismo sumador y solo cambian dos señales de entrada:

sel	b_signal	ci	result
'0' (add)	b	'0'	a + b
'1' (sub)	not b	'1'	a - b

Figura 1: Tabla de valores del `AdderSub8`: suma y resta compartiendo el mismo `Adder8`.

Las dos filas de esta tabla son, literalmente, dos expresiones `with/select`: una que elige entre `b` y `not b`, y otra que elige entre `'0'` y `'1'` para el acarreo de entrada. Con esas dos señales listas, se instancia el `Adder8` y su salida `s` es el resultado.
Como el resultado tiene el mismo ancho que los operandos (8 bits), las operaciones se interpretan módulo 256: si la suma excede el valor máximo, el resultado conserva únicamente los 8 bits menos significativos, y la resta de un valor mayor entrega el complemento a dos correspondiente. Ese comportamiento es el esperado y no debe corregirse; el bit que se pierde es el acarreo de salida `co` del `Adder8`, que en la Actividad 2 alimentará la *flag* `c`.

Componentes entregados

Los siguientes componentes vienen descritos en el proyecto base. **No deben modificarse ni volver a describirse**: se importan al proyecto, se declaran donde se necesiten y se instancian con `port map`.

Aritmética: `Adder8`

Suma dos vectores de 8 bits más un acarreo de entrada, y entrega la suma de 8 bits junto con el acarreo de salida. Internamente son ocho instancias de `FA`, y cada `FA` son dos instancias de `HA`: no hay ningún operador aritmético involucrado, solo compuertas. Al importarlo al proyecto deben incluirse también `FA.vhd` y `HA.vhd`, o Vivado no podrá elaborar la jerarquía.

Registro: `Reg`

Un registro es un pequeño bloque de memoria dentro del circuito que almacena datos. Recibe, guarda y envía datos coordinado por un reloj: la señal `clock` es la entrada del reloj, y el registro solo actúa en su **flanco de subida**, es decir, en el instante en que la señal pasa de nivel bajo a nivel alto. El `Reg` entregado es además un **contador**: puede reiniciarse a cero con `clear` e incrementar o decrementar su valor con `up` y `down`.

Entradas						Salidas
`clock`	`clear`	`load`	`up`	`down`	`datain(7:0)`	`dataout(7:0)`
*	1	*	*	*	*	"00000000"
Flanco Subida	0	1	*	*	*	datain
Flanco Subida	0	0	1	*	*	dataout + 1
Flanco Subida	0	0	0	1	*	dataout - 1
*	*	*	*	*	*	dataout

Figura 2: Tabla de valores de `Reg`. Las filas se leen en orden: la primera que aplica es la que manda.

La tabla se lee así: `clear` tiene prioridad y vuelve la salida a cero sin necesidad de flanco; con `clear` en `'0'` y `load` en `'1'`, en el flanco de subida se guarda `datain`; si no hay flanco, o ninguna señal de control está activa, el registro conserva su valor. En esta actividad basta con usar `load` fijo en `'1'` y controlar **cuándo** se carga a través de la señal conectada a `clock`.

Salida: `Display_Controller`

Las salidas de un circuito que representan un número binario usando LEDs no escalan bien. Una alternativa es ordenar los LEDs para que formen figuras con significado, por ejemplo 7 LEDs que representen

los segmentos de un dígito hexadecimal. Esto mejora la legibilidad, pero aumenta las señales necesarias. Sin embargo, como la vista tiene persistencia, es posible rotar rápidamente entre distintos dígitos y obtener la ilusión de que funcionan simultáneamente, usando así menos señales. Para hacer esa rotación, el componente recibe una señal de un *clock* con una frecuencia conocida.

- `Display_Controller.vhd`

  Este módulo recibe cuatro vectores de 4 bits (`dis_a`, `dis_b`, `dis_c` y `dis_d`) y un valor lógico `clk`. Entrega como salida los vectores `seg[7:0]` y `an[3:0]`.

  Su función es decodificar los valores que se quieren entregar en los *display* de 7 segmentos correspondientes:



  Para que este módulo funcione, debe conectar la señal de entrada de la placa Basys3 `clk` a la señal de entrada `clk` de este módulo. Su salida genera las señales de salida de la placa Basys3 `seg[7:0]` y `an[3:0]` para encender el *display*.

Entrada: `Debouncer`

Las entradas de un circuito pueden no ser lo suficientemente estables. Por ejemplo, al apretar un botón la señal tiende a rebotar antes de estabilizarse en su nuevo valor.

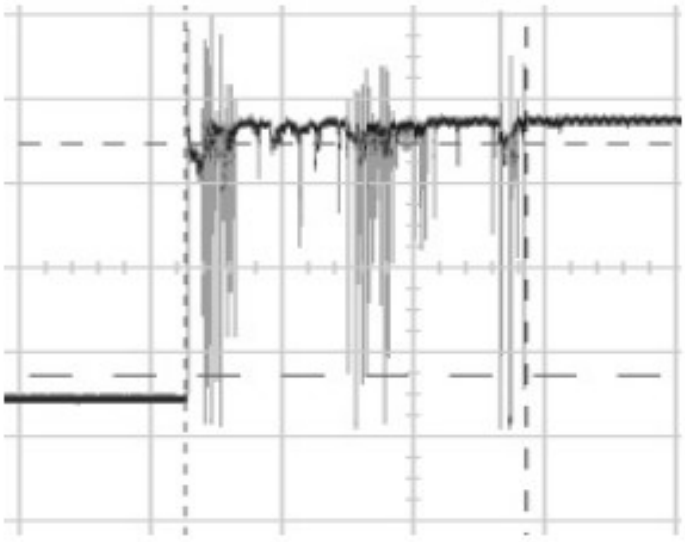


Figura 3: Señal al pulsar un botón.

Una solución es tener un circuito que no transfiera la señal hasta que esta sea estable por un periodo de tiempo. A ese componente lo llamaremos *debouncer* y, para permitirle medir el tiempo, le daremos una señal de un *clock* con una frecuencia conocida.

Importante: la señal `btn` no debe usarse como reloj de un registro. Cada rebote es un flanco de subida y el registro cargaría varias veces con una sola pulsación. Debe usarse siempre la señal que sale del `Debouncer`, que por conveniencia llamaremos `d_btn`.

Declaraciones

Las declaraciones de los componentes entregados son las siguientes:

```
-- Declaraciones de los componentes entregados en el proyecto base.
-- No deben describirse: basta con importarlos, declararlos e instanciarlos.

component Adder8 -- No Tocar
    Port (
        a  : in  std_logic_vector (7 downto 0);
        b  : in  std_logic_vector (7 downto 0);
        ci : in  std_logic;
        s  : out std_logic_vector (7 downto 0);
        co : out std_logic
    );
end component;

component Reg -- No Tocar
    Port (
        clock   : in  std_logic;
        clear   : in  std_logic;
        load    : in  std_logic;
        up      : in  std_logic;
        down    : in  std_logic;
        datain  : in  std_logic_vector (7 downto 0);
        dataout : out std_logic_vector (7 downto 0)
    );
end component;

component Display_Controller -- No Tocar
    Port (
        dis_a : in  std_logic_vector (3 downto 0);
        dis_b : in  std_logic_vector (3 downto 0);
        dis_c : in  std_logic_vector (3 downto 0);
        dis_d : in  std_logic_vector (3 downto 0);
        clk   : in  std_logic;
        seg   : out std_logic_vector (7 downto 0);
        an    : out std_logic_vector (3 downto 0)
    );
end component;

component Debouncer -- No Tocar
    Port (
        clk        : in  std_logic;
        signal_in  : in  std_logic;
        signal_out : out std_logic
    );
end component;
```

Tareas

Usando el proyecto base entregado, se espera que realice lo siguiente:

- Probar la implementación del proyecto base cargando el binario en la FPGA. Es decir, en Vivado proceder con la secuencia: `Run Synthesis` >`Run Implementation` >`Generate Bitstream` >`Program Device`

- **Copiar e importar** al proyecto base todos los componentes entregados, incluyendo `HA` y `FA`, de los que depende el `Adder8`.
- Crear y definir la **entidad AdderSub8**, con entradas `a(7:0)`, `b(7:0)` y `sel`, y salida `result(7:0)`. Debe implementar la tabla de valores de la Figura 1 **instanciando el `Adder8`** entregado.
- Reparar las instancias `inst_REG_A` e `inst_REG_B` para que cada registro cargue con su botón, e instanciar el `AdderSub8` en `Basys3.vhd`, de modo que las conexiones concuerden con los diagramas de las Figuras 4 y 5.
- Probar su implementación cargando el binario en la FPGA.

¿Cuál es el producto final de esta actividad?

Al finalizar la actividad, se espera que los diagramas de las Figuras 4 y 5 estén descritos en VHDL y puedan ser probados en la placa durante el horario de laboratorio por un ayudante sin ningún tipo de error. El resultado es una calculadora capaz de acumular su resultado: el *muxer* de entrada elige si los registros se cargan con los interruptores o con el resultado del sumador-restador.

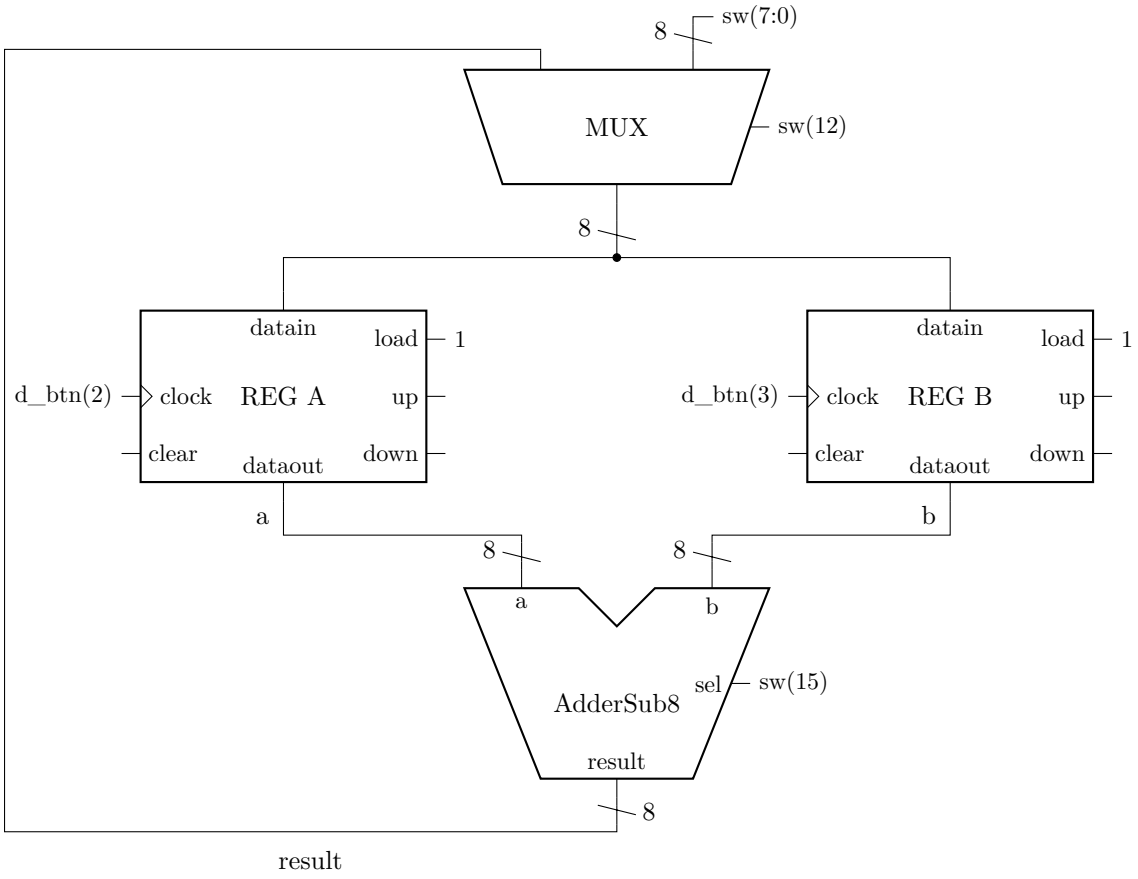


Figura 4: Diagrama de la calculadora con acumulación de resultado.

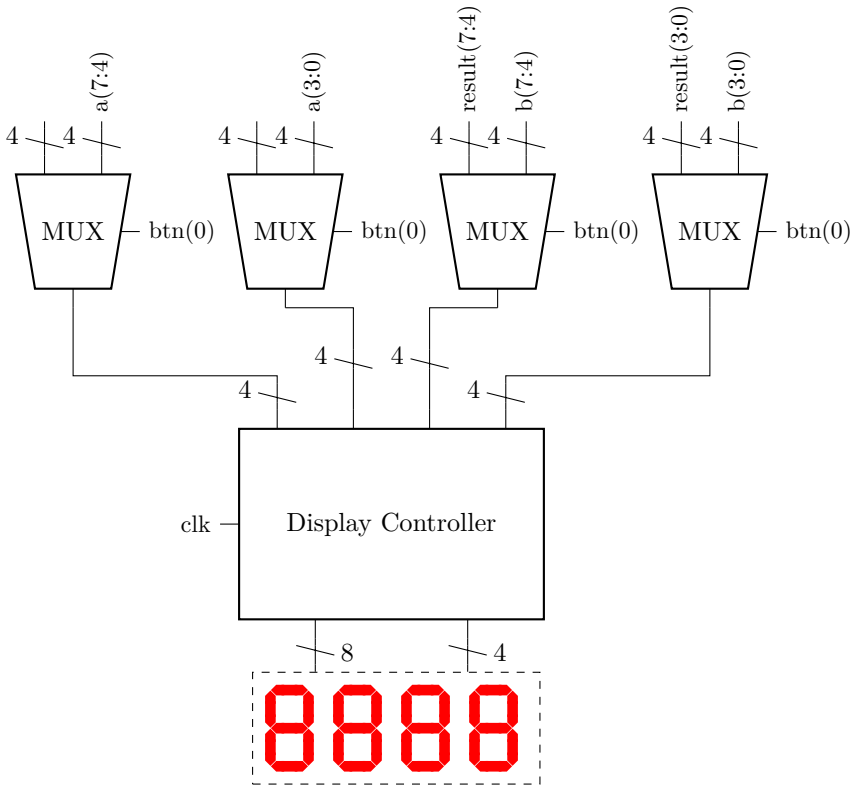


Figura 5: Diagrama de las conexiones de los *display* de 7 segmentos.

Para verificar el funcionamiento conviene recorrer al menos los siguientes casos:

- Cargar `a` = 3 y `b` = 5 desde los interruptores. Con `sw(15)` en `'0'` el *display* debe mostrar `08`; al subirlo a `'1'`, `FE` (3 − 5 en complemento a dos).
- Con `a` = `b`, la resta debe mostrar `00`.
- Con `a` = `FF` y `b` = `01`, la suma debe mostrar `00` (desborde) y la resta `FE`.
- Cambiar `sw(12)` para cargar un registro con el resultado en vez de con los interruptores, y comprobar que la calculadora acumula.

Si la suma es correcta y la resta no, el problema casi siempre está en el complemento a dos: falta el +1 que entra por `ci`, o se invirtió el operando equivocado.